Serialización/deserialización de datos

Agustín C. Caminero Herráez
Depto. de Sistemas de Comunicación y Control.
UNED
accaminero@scc.uned.es / @accaminero

ÍNDICE

- ¿Qué es serialización/deserialización de datos?
- Avro
- Parquet

OBJETIVOS

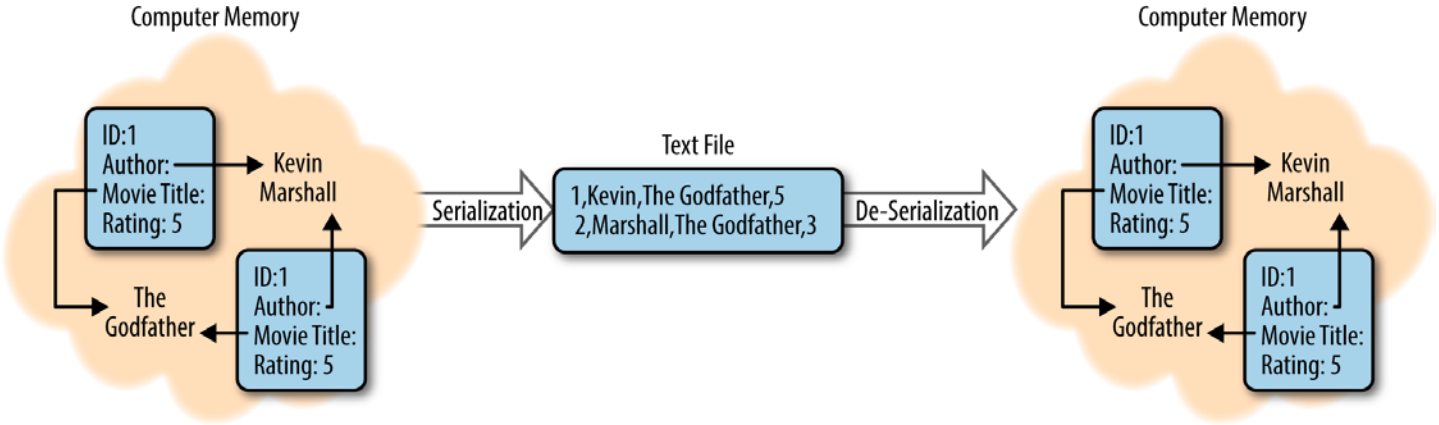
- Conocer distintos tipos de serializadores/deserializadores (serdes).
- Comprender cuándo es necesario cada uno.
- Comprender la estructura del archivo para cada serde.

¿QUÉ ES SERIALIZACIÓN/DESERIALIZACIÓN DE DATOS?

- Las aplicaciones trabajan normalmente con al menos dos representaciones de datos:
  - En **memoria**, los datos se almacenan en objectos, structs, listas, arrays, árboles, tablas hash, …
    - Estas estructuras de datos se encuentran optimizadas para su acceso y manipulación eficiencite por parte de la CPU (típicamente utilizando punteros).
  - Cuando se desea almacenar datos en ficheros o enviarlos a través de la red, se tienen que codificar utilizando alguna **secuencia de bytes** autocontenida (por ejemplo, un documento JSON).
    - Esta representación es muy diferente de las estructuras de datos que se utilizan en memoria.

¿QUÉ ES SERIALIZACIÓN/DESERIALIZACIÓN DE DATOS?

- Por tanto, es necesario realizar la traducción entre ambas representaciones.
  - Desde memoria a bytes → **serialización** (o codificación, marshalling).
  - Desde bytes a memoria → **deserialización** (o decodificación, unmarshalling).
- Hay una gran variedad de propuestas.



FORMATOS DE CODIFICACIÓN DE DATOS

- Cada lenguaje de programación tiene su propio soporte para serialización/deserialización (**serdes**), pero tienen problemas:
  - Están fuertemente ligadas a su lenguaje.
  - A menudo no tienen en cuenta la compatibilidad hacia adelante y atrás.
  - Eficiencia.
- No es buena idea utilizar el serde de un lenguaje, salvo para casos muy transitorios.

FORMATOS DE CODIFICACIÓN DE DATOS

- Aspectos a considerar para elegir un formato de datos:
  - Tamaño de los datos.
  - Velocidad de lectura/escritura.
  - Legibilidad por humanos (human readability).
  - Facilidad de uso.

FORMATOS DE CODIFICACIÓN DE DATOS

- Formatos textuales:
  - JSON, XML, CSV
  - Problemas:
    - Codificación de números, soporte para esquemas …
- Codicación binaria:
  - Basadas en JSON:
    - MessagePack, BSON, BJSON, UBJSON, BISON, Smile…
  - Basadas en XML:
    - WBXML, Fast Infoset …
  - Avro, Parquet, Protobuf, Thrift, ORC, SequenceFile…

FORMATOS DE CODIFICACIÓN DE DATOS

- Dos de los formatos más ampliamente utilizados en el ecosistema de Hadoop son Avro y Parquet.
  - Avro está basado en filas.
  - Parquet está basado en columnas.
- Comparaciones:
  - http://blog.cloudera.com/blog/2016/04/benchmarking-apache-parquet-the-allstate-experience/
  - https://db-blog.web.cern.ch/blog/zbigniew-baranowski/2017-01-performance-comparison-different-file-formats-and-storage-engines





AVRO



- Formato binario de codificación de datos.
- Avro utiliza un esquema para especificar la estructura de los datos que se están codificando.
- Utiliza dos lenguajes para definir esquemas:
  - Avro IDL: para edición humana.
  - Otro basado en JSON: legible por máquinas.
- Para parsear los datos binarios, tenemos que recorrer los campos en el orden en que aparecen en el esquema para saber el tipo de dato de cada campo.
  - El código que generó los datos debe utilizar el mismo esquema que el código que los lee.
- Soporta compresión de datos con los algoritmos Snappy y deflate.
- Es un proyecto complejo, que incluye entre otras funcionalidades Remote procedure call (RPC).

AVRO: ESQUEMA

- Ejemplo:
  - Datos en JSON:

```
{
    "userName": "Martin",
    "favoriteNumber": 1337,
    "interests": ["daydreaming", "hacking"]
}
```

AVRO: ESQUEMA

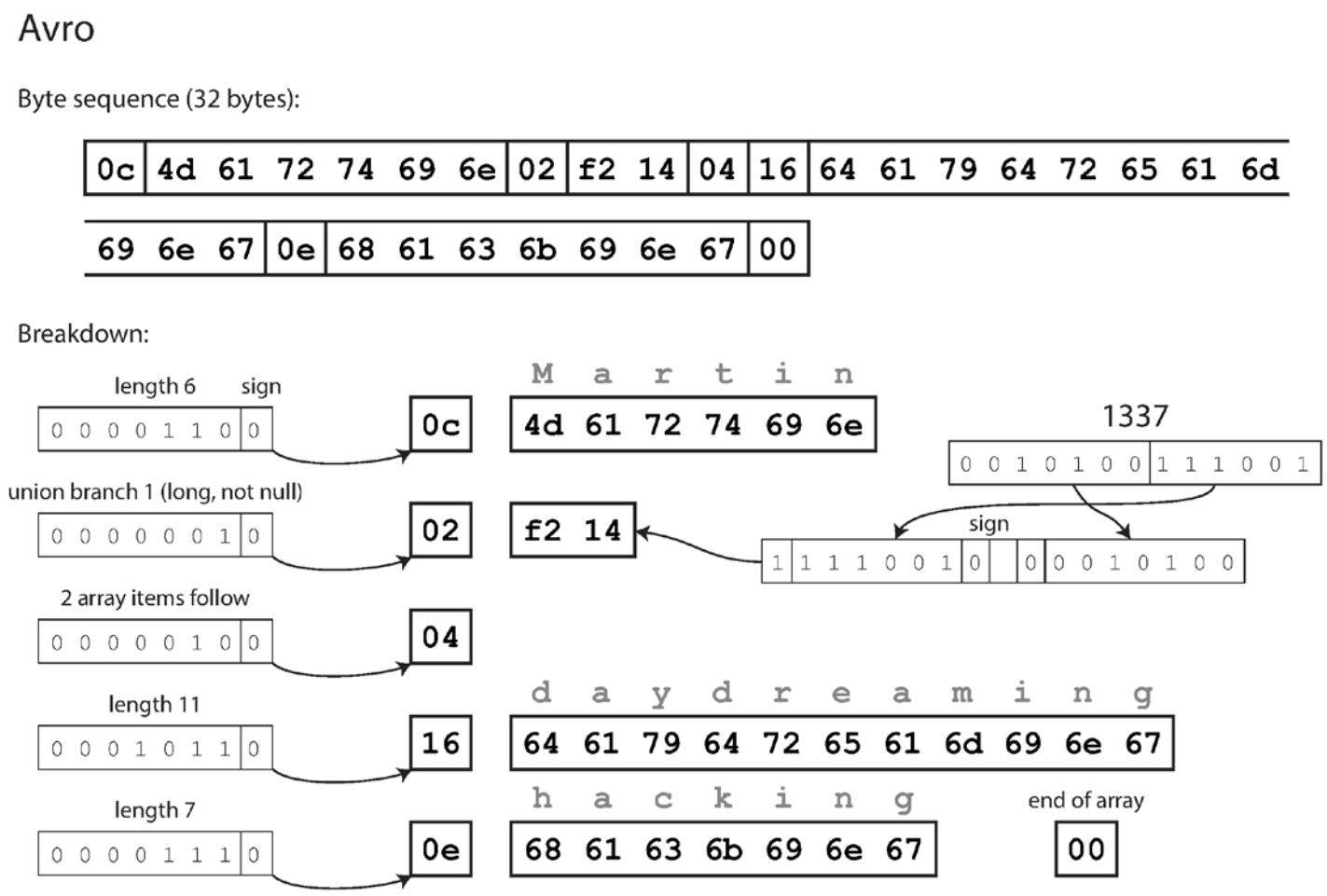
- Esquema en Avro IDL:

```
record Person {
    string               userName;
    union { null, long }
favoriteNumber = null;
    array<string>
interests;
}
```

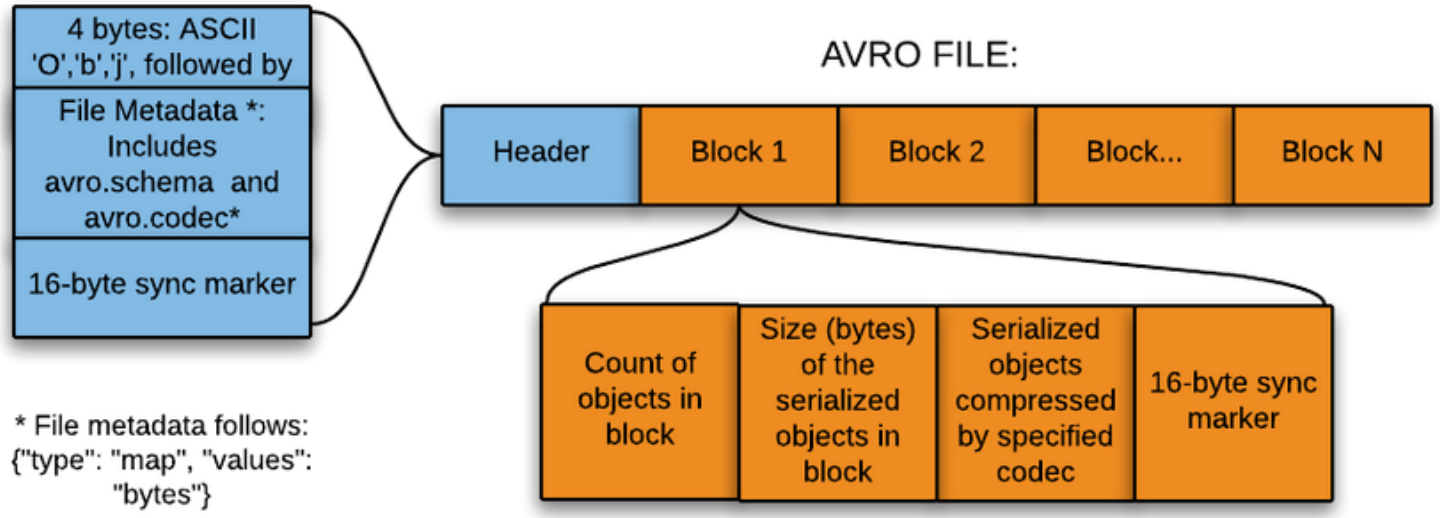
- Esquema en JSON:

```
{
    "type": "record",
    "name": "Person",
    "fields": [
        {"name": "userName",
"type": "string"},
        {"name": "favoriteNumber",
"type": ["null", "long"], "default":
null},
        {"name": "interests",      "type":
{"type": "array", "items": "string"}}
    ]
}
```

AVRO: ESQUEMA



AVRO: ESTRUCTURA DE ARCHIVO



AVRO: EVOLUCIÓN DE ESQUEMAS

- **¿Es necesario que el esquema usado para escribir esté disponible en el momento de leer los datos?**
- Conceptos:
  - **Esquema de escritura**: es el esquema utilizado para escribir un archivo.
  - **Esquema de lectura**: es el esquema utilizado para leer un archivo.
  - **Compatibilidad hacia adelante:** esquema de lectura viejo puede leer datos escritos con esquema de escritura nuevo.
  - **Compatibilidad hacia atrás:** esquema de lectura nuevo puede leer datos escritos con esquema de escritura viejo.
- No es necesario que el esquema de escritura y el de lectura sean el mismo, basta con que sean compatibles.
  - ¿Qué significa que sean compatibles? → Solamente se pueden añadir o eliminar datos que tengan valores por defecto.
- Cuando se decodifican (leen) los datos, la librería de Avro resuelve las diferencias mirando el esquema de lectura y de escritura y traduciendo del esquema de escritura al esquema de lectura.

AVRO: EVOLUCIÓN DE ESQUEMAS

- Situaciones:
  - Campos en distinto orden en el esquema de escritura y en el del lectura no son problema.
  - Campo existente en el esquema de escritura pero no en el de lectura: Se ignora ese campo.
  - Campo existente en el esquema de lectura pero no en el de escritura: Se devuelve un valor por defecto declarado en el esquema de lectura.
  - Campo existente en ambos esquemas pero con distinto tipo: no hay problema si Avro puede convertir el tipo.
  - Campo con distinto nombre: Es posible pero…
    - Si el esquema de lectura tiene un alias que contenga el nombre del campo en el esquema de escritura no hay problema.
    - Es compatible hacia atrás pero no hacia adelante.
  - Añadir una nueva rama a una unión es compatible hacia atrás pero no hacia adelante.

AVRO: EVOLUCIÓN DE ESQUEMAS

Writer's schema for Person record

Datatype	Field name
string	userName
union {null, long}	favoriteNumber
array<string>	interests
string	photoURL

Reader's schema for Person record

Datatype	Field name
long	userID
union {null, int}	favoriteNumber
string	userName
array<string>	interests

AVRO: ESQUEMA

- **¿Cómo sabe el lector cuál es el esquema de escritura de unos datos?**
- Depende del contexto:
  - Un gran fichero con muchos registros: El esquema de escritura puede ir al principio del fichero.
  - Base de datos con registros individuales: Es posible que registros diferentes se hayan escrituo con esquemas distintos, por lo que en cada registro se incluye la versión del esquema que se ha utilizado, y la base de datos puede mantener un listado con las versiones.
  - Mandar registros por red: Los procesos involucrados en la comunicación pueden negociar la versión del esquema durante el establecimiento de la conexión.
- Mantener una base de datos con las versiones de los esquemas es buena idea en cualquier caso.

AVRO: TIPOS DE DATOS

Type	Description	Schema
`null`	The absence of a value	`"null"`
`boolean`	A binary value	`"boolean"`
`int`	32-bit signed integer	`"int"`
`long`	64-bit signed integer	`"long"`
`float`	Single-precision (32-bit) IEEE 754 floating-point number	`"float"`
`double`	Double-precision (64-bit) IEEE 754 floating-point number	`"double"`
`bytes`	Sequence of 8-bit unsigned bytes	`"bytes"`
`string`	Sequence of Unicode characters	`"string"`

AVRO:TIPOS DE DATOS

Type	Description	Schema example
`array`	An ordered collection of objects. All objects in a particular array must have the same schema.	`{ "type": "array", "items": "long" }`
`map`	An unordered collection of key-value pairs. Keys must be strings and values may be any type, although within a particular map, all values must have the same schema.	`{ "type": "map", "values": "string" }`

AVRO: TIPOS DE DATOS

Type	Description	Schema example
`record`	A collection of named fields of any type.	`{ "type": "record", "name": "WeatherRecord", "doc": "A weather reading.", "fields": [ {"name": "year", "type": "int"}, {"name": "temperature", "type": "int"}, {"name": "stationId", "type": "string"} ] }`
`enum`	A set of named values.	`{ "type": "enum", "name": "Cutlery", "doc": "An eating utensil.", "symbols": ["KNIFE", "FORK", "SPOON"] }`

AVRO: TIPOS DE DATOS

Type	Description	Schema example
`fixed`	A fixed number of 8-bit unsigned bytes.	`{ "type": "fixed", "name": "Md5Hash", "size": 16 }`
`union`	A union of schemas. A union is represented by a JSON array, where each element in the array is a schema. Data represented by a union must match one of the schemas in the union.	`[ "null", "string", {"type": "map", "values": "string"} ]`

AVRO

- Cloudera proporciona Avro con una serie de herramientas de utilidad:
  - Pasar de JSON a avro, de avro a JSON, a texto, obtener el esquema, …
  - Ejecutamos desde consola:
    - java -jar /usr/lib/avro/avro-tools-1.7.6-cdh5.7.0.jar
- Trabajo con Avro:
  - Avro-hive-estudiantes.ipynb

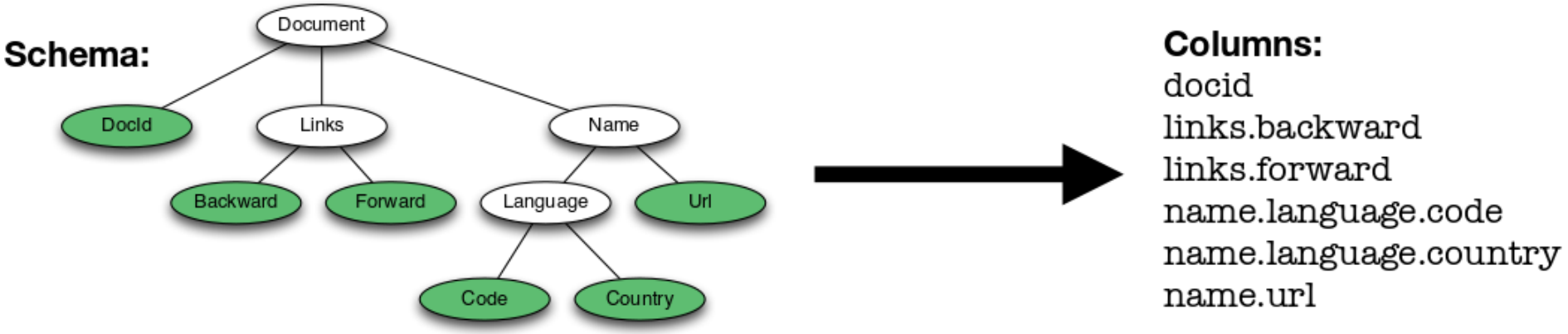
PARQUET



- Formato de almacenamiento de datos en columnas:
  - Los datos de cada fila correspondientes a la misma columna se almacenan juntos.
  - Permite alcanzar tamaños de ficheros más pequeños que los archivos orientados a filas.
    - Una columna que almacene un timestamp se puede codificar almacenando el primer valor y luego las diferencias entre los valores siguientes.

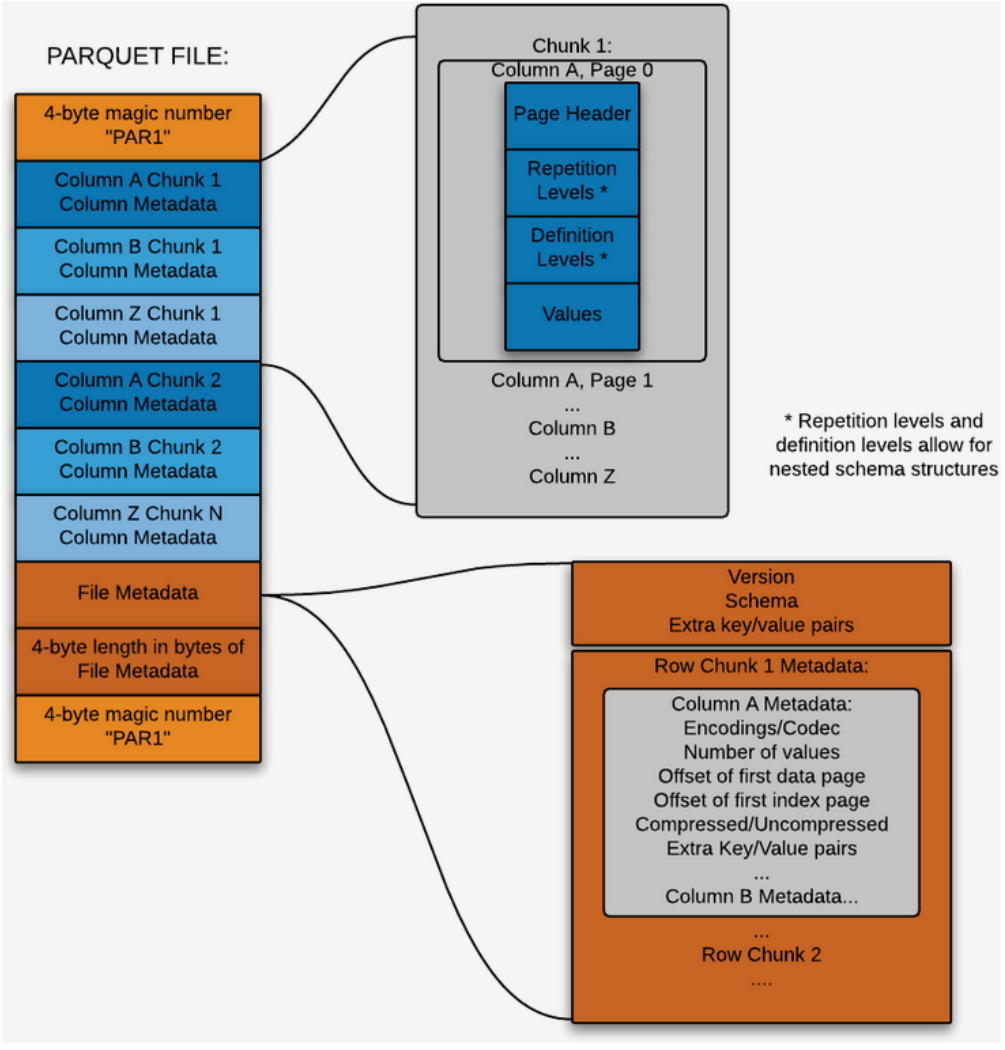
PARQUET

- Permite almacenar estructuras de datos anidadas.



PARQUET: ESTRUCTURA DE ARCHIVO

- Header:
  - Constante de 4 bytes que identifica este archivo como de tipo Parquet (PAR1).
- Footer:
  - Incluye los metadatos del archivo.
    - Version, esquema, longitud del footer. … acaba con PAR1
- Column chunk:
  - Pedazos de columnas. Los datos para esos column chunks están almacenado en páginas.
- Page:
  - Cada página almacena valores de una columna.
  - Tienen valores similares por lo que son buenos candidatos para comprimir.



PARQUET: COMPRESIÓN

- El primer nivel de compresión se consigue definiendo la forma en que se codifican los datos.
  - La codificación más sencilla es escribir los datos completos (ej, un int32 se escribe como 4 bytes).
  - Parquet permite codificaciones más compactas, como por ejemplo:
    - Codificación delta: almacena diferencias entre valores.
    - Codificación run-length: valores repetidos se codifican como un único valor y la cuenta de repeticiones.
    - Codificación de diccionario: se construye un diccionario de valores y luego se codifican los valores como enteros que representan los índices del diccionario.
    - Bit packing: almacena varios valores pequeños en un único byte.
  - Cuando se escribe un fichero, Parquet elegirá la codificación más adecuada de forma automática basándose en el tipo de la columna.
    - Por ejemplo, boolean se puede codificar como una combinación de run-length y bit packing.
- Otro nivel de compresión consiste en aplicar un algoritmo estándar de compresión (Snappy, gzip o LZO).

PARQUET: TIPOS DE DATOS

Type	Description
`boolean`	Binary value
`int32`	32-bit signed integer
`int64`	64-bit signed integer
`int96`	96-bit signed integer
`float`	Single-precision (32-bit) IEEE 754 floating-point number
`double`	Double-precision (64-bit) IEEE 754 floating-point number
`binary`	Sequence of 8-bit unsigned bytes
`fixed_len_byte_array`	Fixed number of 8-bit unsigned bytes

PARQUET: TIPOS DE DATOS

Logical type annotation	Description	Schema example
`UTF8`	A UTF-8 character string. Annotates `binary`.	`message m { required binary a (UTF8); }`
`ENUM`	A set of named values. Annotates `binary`.	`message m { required binary a (ENUM); }`
`DECIMAL(precision,scale)`	An arbitrary-precision signed decimal number. Annotates `int32`, `int64`, `binary`, or `fixed_len_byte_array`.	`message m { required int32 a (DECIMAL(5,2)); }`
`DATE`	A date with no time value. Annotates `int32`. Represented by the number of days since the Unix epoch (January 1, 1970).	`message m { required int32 a (DATE); }`

PARQUET: TIPOS DE DATOS

Logical type annotation	Description	Schema example
`LIST`	An ordered collection of values. Annotates `group`.	`message m { required group a (LIST) { repeated group list { required int32 element; } } }`
`MAP`	An unordered collection of key-value pairs. Annotates `group`.	`message m { required group a (MAP) { repeated group key_value { required binary key (UTF8); optional int32 value; } } }`

PARQUET

- Cloudera proporciona Parquet con una serie de herramientas de utilidad:
  - Obtener el esquema, mostrar un fichero en JSON, …
  - Ejecutamos desde consola:
    - parquet-tools --help
- Trabajo con Parquet:
  - Parquet-hive-estudiantes.ipynb

Serialización/deserialización de datos

Agustín C. Caminero Herráez
Depto. de Sistemas de Comunicación y Control.
UNED
accaminero@scc.uned.es / @accaminero